Software Requirements Specification

for

ZePRO APP (UG/PG Project Allocation)

Version 1.0 approved

Prepared by Team 7

Konki Navadeep – B230386CS

Yangala Reddy Sekhar – B231346CS

Shaik Mehtab – B230703CS

Chilakala Nagendra Reddy – B230650CS

Software Engineering Lab

12th January 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Revision History **3**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2
2. Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 3
2.3 User Classes and Characteristics 4
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements **7**
3.1 User Interfaces 7
3.2 Hardware Interfaces 9
3.3 Software Interfaces 10
3.4 Communications Interfaces 10
4. System Features **11**
4.1 User Registration & Authentication 11
4.2 Institution, Department & Coordinator Management 12
4.3 Project Slot & Allocation Rule Management 12
4.4 Faculty Profile & Slot Management 12
4.5 Faculty Domain, Sub-Domain & Problem Statement Selection 13
4.6 Project Filtering and Search 13
4.7 Student Profile, Team Formation & Document Upload 14
4.8 Faculty Request Handling & Approval Workflow 14
4.9 Notification and Email Communication 15
4.10 Coordinator Dashboard & Allocation Reports 15
4.11 Administration & Record Management 16
5.Other Nonfunctional Requirements **16**
5.1 Performance Requirements 16
5.2 Safety Requirements 17
5.3 Security Requirements 17
5.4 Software Quality Attributes 18
5.5 Business Rules 18
6. Other Requirements **19**
Appendix A: Glossary **19**

Appendix B: Analysis Models **20**
Appendix C: To Be Determined List **20**

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

*The **UG/PG Project Allocation System** is an application developed using **Flutter and Firebase** to provide a structured and streamlined workflow for academic project allocation across multiple colleges and branches. Administrators manage colleges and departments, and students register under their respective college and branch using secure authentication. This SRS document outlines the scope and requirements of the system.*

1.2 Document Conventions

- *Section headings are numbered hierarchically (e.g., 1, 1.1, 1.1.1) to clearly represent the structure and organization of the requirements.*
- *The keywords **shall** and **may** are used consistently in accordance with IEEE standards, where:*
    - ***Shall** indicates a mandatory requirement*
    - ***May** indicates an optional requirement*
- *Important terms, system modules, and features are highlighted in **bold** when they are first introduced.*
- *User roles such as **Admin**, **Faculty**, **Student**, and **Coordinator** are capitalized to distinguish them from general terminology*

1.3 Intended Audience and Reading Suggestions

- *Students (UG/PG):*
  Undergraduate and postgraduate students who use the system to view available projects, track allocation status, and communicate with faculty. Students should focus on the Overall Description and Functional Requirements sections relevant to student activities.
- *Faculty Members:*
  Faculty members who propose projects, review student requests, guide students. Faculty readers should review the Overall Description and the functional requirements related to project creation and approval.
- *Faculty Coordinators:*
  Coordinators responsible for managing project allocation workflows, approvals, and conflict resolution. They should carefully read the System Features, Functional Requirements.
- *Administrators:*
  System administrators who manage users, roles, system configuration, and maintenance. Administrators should review the System Overview, Non-Functional Requirements, and Administrative Functional Requirements sections.

- *Developers:*
  Software developers responsible for designing and implementing the system. Developers should read the document sequentially, with particular emphasis on the Functional Requirements, Non-Functional Requirements, and System Interfaces sections.

1.4 Product Scope

- *The UG/PG Project Allocation System is an application developed to automate and manage the allocation of academic projects to undergraduate and postgraduate students.*
- *Its primary purpose is to reduce manual effort, improve efficiency, and ensure a fair and transparent project allocation process.*
- *The system benefits Students, Faculty, Coordinators, and Administrators by streamlining project selection, approval, and management activities.*
- *This software supports institutional academic goals by improving project management accountability, and overall academic workflow efficiency.*

1.5 References

- ***Software Requirements Specification (SRS) Standard***
  IEEE Std 29148-2018
- ***Flutter Framework Documentation***
  Flutter SDK Documentation
- ***Firebase Documentation***
  Firebase Authentication, Cloud Firestore, and Cloud Messaging Documentation

- ***Institutional Academic Policies***
  UG/PG Project Allocation and Supervision Guidelines
  Author: Participating Institutions / Departments
  Source: Institutional policy documents

2. Overall Description

2.1 Product Perspective

- *This product is not part of an existing product family and is being created as a fresh solution to address limitations of current manual or informal allocation practices.*
- *The application is a self-contained system that provides functionality for project proposal, allocation management, and vacancy tracking.*

- *It interacts directly with external users such as Students, Faculty, Faculty Coordinators, and Administrators through dedicated user interfaces, without dependency on other software systems.*

2.2 Product Functions

The application must facilitate the following key functions:

1. User Registration and Authentication

Users must be able to securely access the application based on their assigned roles.

- ***Signup and Login:*** *Users register and log in using authorized credentials.*
- ***Role Based Access:*** *Students, Faculty, Faculty Coordinators, and Administrators have different access rights.*
- ***Profile Management:*** *Users can view and update basic personal and academic details.*
- ***Password Recovery:*** *Secure password reset functionality is provided.*

2. Project Creation and Management

Faculty members manage academic project information through the application.

- ***Project Proposal:*** *Faculty can create and publish UG/PG project topics.*
- ***Project Details Management:*** *Project title, description, domain, and availability are maintained.*
- ***Project Visibility:*** *Project slots are displayed to eligible students based on criteria.*

3. Student Project Selection

Students interact with the system to choose projects.

- ***Project Browsing:*** *Students can view available project listings.*
- ***Request Submission:*** *Students can submit and update project requests within a defined period.*
- ***Allocation Status:*** *Students can view their assigned project details.*

4. Project Allocation Processing

The system manages fair and transparent project allocation.

- ***Allocation Rules:*** *Projects are allocated based on predefined rules and availability of slots and meeting outcomes.*
- ***Coordinator Control:*** *Faculty Coordinators oversee and finalize the allocation process.*

- ***Conflict Handling:*** *The system supports resolution of allocation conflicts by scheduling meeting for all the requests by students.*

5. Supervision

The application supports academic supervision activities.

- ***Faculty Supervision:*** *Faculty can view and manage assigned students.*
- ***Faculty Coordinator Supervision:*** *Faculty Coordinator can view remaining non-allocated groups/students and faculty who are free.*
- ***Supervision Panel:*** *This panel is visible to Faculty Coordinator to view the status of students and faculty.*

6. Administration and Maintenance

Administrative functions ensure smooth operation of the system.

- ***User Management:*** *Administrators manage users and roles.*
- ***System Maintenance:*** *Basic configuration and data maintenance features are provided.*
- ***Record Management:*** *The system maintains accurate records of projects and allocations.*

2.3 User Classes and Characteristics

The UG/PG Project Allocation System is intended to be used by different classes of users with varying roles, access privileges, and levels of interaction. The major user classes are described below:

- ***Students (UG/PG)***
  Students are the primary users of the system. They use the application to view available projects, submit project requests, and track allocation status. Students have basic technical knowledge and limited access privileges restricted to student related functionalities.
- ***Faculty Members***
  Faculty members use the system to propose project domains and sub-domains, review student requests, and lock allocated student slots. They interact with the system and have moderate technical expertise with access limited to faculty related features.
- ***Faculty Coordinators***
  Faculty coordinators are key users responsible for overseeing the project allocation process and setting the rules for project allocation. This user class has higher privileges and frequently interacts with students as they have a separate dashboard for viewing the non-allocated students and the faculties who are free to allocate students under them.

- ***Administrators***
  Administrators manage user accounts, roles, system configuration, and maintenance activities. They have the highest level of access and technical expertise but use the system less frequently compared to coordinators.

2.4 Operating Environment

- *Platform: Android*
- *Operating System: Compatible with Android 8.0 and above.*
- *Development Framework: Flutter for cross platform UI development.*
- *Backend Services: Firebase for authentication, database, cloud storage, and notifications.*
- *Database: Firestore (NoSQL)*
- *Network: Internet / Institutional LAN / WIFI*
- *Software Dependencies: Google Play Services for Firebase integration, stable internet connectivity for data synchronization and authentication.*

2.5 Design and Implementation Constraints

- ***Platform Constraint***
  - *The application shall be developed exclusively as an **Android mobile application** and must support **Android 8.0 and above**.*
- ***Technology Constraint***
  - *The system shall be developed using **Flutter** as the application framework.*
  - ***Firebase** shall be used for backend services such as authentication, database storage, and cloud services.*
- ***Hardware Constraints***
  - *The application must operate efficiently on standard Android devices with limited memory and processing power.*
  - *Performance should be optimized for mid-range smartphones.*
- ***Network Constraint***
  - *A stable **internet connection** is required for authentication, data synchronization, and project allocation operations.*
- ***Security Constraints***
  - *User authentication and data access must be secured using Firebase security rules and role-based access control.*

- *Sensitive user data must be protected against unauthorized access.*

- ***Maintenance and Standards Constraint***
  - *The application should follow standard **Flutter coding conventions** and clean architecture practices to support future maintenance and scalability*

2.6 User Documentation

The mentioned documentations will be provided later:

- *Student User Manual*
- *Faculty User Manual*
- *Department Coordinator Guide*
- *System Administrator Guide*
- *In-app Help and Tooltips*

2.7 Assumptions and Dependencies

Assumptions

- *It is assumed that all users (**Students, Faculty, Coordinators, and Administrators**) have access to compatible **Android devices** and basic knowledge of using mobile applications.*
- *It is assumed that users will have a **stable internet connection** to access system features such as authentication, data synchronization, and project allocation.*
- *The system assumes the continued availability and support of **Flutter** and **Firebase** technologies during development and deployment.*
- *It is assumed that institutions using the application will follow a **standard project allocation process**, enabling the system to be used without major customization.*

Dependencies

- *The application depends on **Firebase services** (Authentication, Database, Cloud Storage) for backend functionality and data management.*
- *The system depends on the **Android operating system (Android (8.0) and above)** for successful deployment and execution.*
- *Proper functioning of the application depends on **Google Play Services** and Firebase infrastructure availability.*
- *Any changes in Firebase policies, pricing, or service availability may impact system functionality.*

3. External Interface Requirements

3.1 User Interfaces

*The system shall provide a **responsive, mobile-based graphical user interface (GUI)** developed using **Flutter**, accessible on **Android devices**.*
*The user interface shall be **role-specific** and dynamically adapt based on the authenticated user's role (**Student, Faculty, Faculty Coordinator, Administrator**).*

General UI Characteristics

- *Clean, intuitive, and consistent layout across all application screens.*
- *Role-based dashboards displayed after successful login.*
- *Standard navigation elements including menu, back navigation, logout option, and help/support section.*
- *Form validation with clear and user-friendly error messages.*
- *Confirmation dialogs for critical actions such as request submission, approval, rejection, and final allocation.*
- *Responsive design to support different Android screen sizes and orientations.*

Student Interface

- *Profile creation and profile update screen.*
- *Document upload interface for marksheets, resume, and required academic documents.*
- *Team creation and team management interface (maximum team size as defined by the coordinator).*
- *Team Leader dashboard displaying:*
    - *Faculty requests sent*
    - *Faculty responses (accepted, rejected, meeting scheduled)*
    - *Current slot availability status*
- *Faculty listing and domain-based project browsing screen.*
- *Notifications panel for request status updates and system alerts.*

Faculty Interface

- *Faculty profile management screen (domains, sub-domains, optional problem statements).*
- *Slot management interface showing total slots and remaining available slots.*
- *Dashboard displaying:*
  - *List of student teams who applied*
  - *Team details (members, documents, department)*
  - *Options to **Accept**, **Reject**, or **Schedule a Meeting***
- *Real-time update of slot availability based on faculty actions.*

Faculty Coordinator Interface

- *Department-level configuration dashboard.*
- *Screens to define and manage allocation rules (maximum team size, slots per faculty).*
- *Allocation monitoring interface showing:*
  - *Non-allocated students*
  - *Faculty slot utilization*
  - *Allocation progress reports*
- *Controls to override rules or reassign slots when required.*

Administrator Interface

- *User and role management screens for students, faculty, and coordinators.*
- *Interfaces to add, remove, or update user records in exceptional cases.*
- *Can change the role of Faculty Coordinator at any time.*
- *System logs and audit trail viewer.*
- *Backup, maintenance, and basic system configuration controls.*

UI Technology

- *The user interface shall be developed using **Flutter**.*
- *The UI shall follow **Android Material Design guidelines** to ensure usability and consistency.*

3.2 Hardware Interfaces

The UG/PG Project Allocation System interacts with the underlying hardware components of Android devices to provide its functionality. The hardware interfaces are described below:

- ***Supported Devices:***
  *The application supports **Android smartphones and tablets** that meet the minimum operating system and hardware requirements.*
- ***Input Devices:***
  *User input is provided through standard mobile hardware components such as **touchscreens**, **on-screen keyboards**, and device navigation buttons.*
- ***Output Devices:***
  *The application displays information using the device **screen** and provides alerts and notifications through **visual indicators**, **sound**, and **vibration**, as supported by the device.*
- ***Network Interface:***
  *The application uses the device's **Wi-Fi or mobile data** to communicate with backend services. Internet connectivity is required for authentication, data synchronization, and real-time updates.*
- ***Notification Interface:***
  *The system uses device notification hardware in conjunction with **Firebase Cloud Messaging (FCM)** to deliver real-time notifications related to project allocation updates and system alerts.*
- ***Data Interaction:***
  All data exchanged between the application and backend services is handled through secure network interfaces without requiring direct access to device specific hardware components.

3.3 Software Interfaces

Component	***Description***
Android Operating System (8.0+)	*Provides the runtime environment for executing the mobile application and accessing device services.*
Flutter SDK	*Used for building the application user interface and handling client-side logic.*
Firebase Authentication	*Manages secure user authentication for Students, Faculty, Coordinators, and Administrators using email and password.*
Firebase Cloud Firestore	*Serves as the primary cloud database for storing user details, project information, preferences, and allocation records in real time.*
Firebase Cloud Messaging (FCM)	*Used to send push notifications such as allocation updates, approvals, and system alerts.*
Firebase Cloud Storage	*Stores related documents or files such as project descriptions or reference materials, if required.*
Google Play Services	*Provides essential background services required for Firebase integration on Android devices.*
RESTful APIs (Firebase SDK)	*Handles communication between the Flutter frontend and Firebase backend services using secure APIs.*
JSON Data Format	*Used for all request and response data exchanged between the application and backend services.*

3.4 Communications Interfaces

The UG/PG Project Allocation System relies on the following communication protocols and interfaces to ensure secure and reliable data exchange between the application and backend services:

- ***HTTPS (SSL/TLS Encryption):***
  *All data exchanged between the mobile application and backend services shall be transmitted securely using **HTTPS** with **SSL/TLS encryption** to protect user data and prevent unauthorized access.*

- ***RESTful APIs:***
  *The application shall use **RESTful APIs** provided by Firebase SDKs for communication between the Flutter frontend and backend services.*
- ***JSON Data Format:***
  *All request and response messages exchanged between the frontend and backend shall use the **JSON (JavaScript Object Notation)** format for structured and lightweight data transfer.*
- ***Firebase Cloud Messaging (FCM):***
  *The system shall use **Firebase Cloud Messaging** to deliver real-time notifications such as project allocation updates, approvals, and system alerts.*
- ***Internet Connectivity:***
  A stable internet connection is required for full application functionality, including authentication, data synchronization, and real-time updates.

4. System Features

4.1 User Registration & Authentication

4.1.1 Description and Priority

Description:
Enables Students, Faculty Members, Faculty Coordinators, and Administrators to create accounts, log in securely, and manage their profiles within the application.

Priority: *High (Essential for system access and role-based functionality)*

4.1.2 Stimulus/Response Sequences

- *User enters login credentials → System validates credentials and grants access.*
- *User requests password change/reset → System verifies identity and updates credentials.*
- *Admin assigns or updates user roles → System applies corresponding permissions.*

4.1.3 Functional Requirements

- ***4.1-1:*** *The system shall support secure user login and logout.*
- ***4.1-2:*** *The system shall support role-based access for Students, Faculty, Faculty Coordinators, and Admins.*
- ***4.1-3:*** *The system shall allow users to update profile details and change passwords.*

4.2 Institution, Department & Coordinator Management

4.2.1 Description and Priority

Description:
Manages multiple institutions and departments, with each department assigned a Project Faculty Coordinator responsible for project allocation rules.

Priority: *High (Core structural feature for multi-institution usage)*

4.2.2 Stimulus/Response Sequences

- *Admin creates an institution or department → System stores and displays structure.*
- *Admin assigns a Faculty Coordinator → System grants coordinator privileges.*
- *Coordinator updates allocation rules → System applies rules to slot management.*

4.2.3 Functional Requirements

- ***4.2-1:*** *The system shall support multiple institutions and departments.*
- ***4.2-2:*** *The system shall allow assignment and modification of Faculty Coordinators.*
- ***4.2-3:*** *The system shall enforce department-specific allocation rules.*

4.3 Project Slot & Allocation Rule Management

4.3.1 Description and Priority

Description:
Allows Faculty Coordinators to define and manage project allocation rules based on ***slot availability****, not random or algorithm-based allocation.*

Priority: *High (Core allocation logic of the application)*

4.3.2 Stimulus/Response Sequences

- *Coordinator sets maximum team size → System validates student group formation.*
- *Coordinator sets faculty slot limits → System updates available slots.*
- *Slot availability changes → System reflects updates across the application.*

4.3.3 Functional Requirements

- ***4.3-1:*** *The system shall allow coordinators to define maximum group size.*
- ***4.3-2:*** *The system shall allow coordinators to define student slot limits per faculty.*
- ***4.3-3:*** *The system shall update slot availability in real time.*

4.4 Faculty Profile & Slot Management

4.4.1 Description and Priority

Description:
Enables faculty members to manage their professional profiles, domains, sub-domains, problem statements, and available student slots.

Priority: *High (Required for student faculty matching)*

4.4.2 Stimulus/Response Sequences

- *Faculty updates slots → System recalculates remaining capacity.*
- *Faculty changes profile/password → System updates stored information.*

4.4.3 Functional Requirements

- ***4.4-1:*** *Faculty shall be able to manage available student slots.*
- ***4.4-2:*** *The system shall reflect slot changes immediately.*

4.5 Faculty Domain, Sub-Domain & Problem Statement Selection

4.5.1 Description and Priority

Description:
Enables faculty members to define their areas of expertise by selecting domains and sub-domains and optionally uploading problem statements for student reference.

Priority: *High*

4.5.2 Stimulus/Response Sequences

Faculty logs in → Faculty selects domains and sub-domains → Faculty uploads or updates problem statement → System saves and displays details.

4.5.3 Functional Requirements

- ***4.5-1:*** *Faculty shall be able to select one or more domains.*
- ***4.5-2:*** *Faculty shall be able to select relevant sub-domains.*
- ***4.5-3:*** *Faculty shall be able to upload or update a problem statement (optional).*
- ***4.5-4:*** *The system shall display faculty domain and problem statement details to students during project browsing.*

4.6 Project Filtering and Search

4.6.1 Description and Priority

Description:
Allows students to efficiently search and filter available projects and faculty based on relevant criteria to identify suitable supervision opportunities.

Priority: *Medium*

4.6.2 Stimulus/Response Sequences

Student selects filter criteria → System updates and displays the filtered project list.

4.6.3 Functional Requirements

- ***4.6-1:*** *The system shall support filtering projects by domain.*
- ***4.6-2:*** *The system shall support filtering by faculty name and department.*
- ***4.6-3:*** *The system shall support filtering by available student slots.*

4.7 Student Profile, Team Formation & Document Upload

4.7.1 Description and Priority

Description:
Allows students to manage profiles, upload required documents, form teams, and request faculty supervision.

Priority: *High (Primary student-side functionality)*

4.7.2 Stimulus/Response Sequences

- *Student logs in → System displays profile and allocation status.*
- *Student uploads documents → System validates and stores files.*
- *Student forms team and decides their team lead → System checks eligibility and deadlines.*

4.7.3 Functional Requirements

- ***4.7-1:*** *Students shall be able to upload marksheets, resumes, and required documents.*
- ***4.7-2:*** *Students shall be able to form teams with a team lead within the defined deadline.*
- ***4.7-3:*** *The system shall restrict team size based on coordinator rules.*

4.8 Faculty Request Handling & Approval Workflow

4.8.1 Description and Priority

Description:
Manages the interaction between student teams and faculty members through request, approval, rejection, or meeting scheduling.

Priority: *High (Controls final allocation outcome)*

4.8.2 Stimulus/Response Sequences

- *Team Lead sends request → System notifies faculty.*
- *Faculty accepts/rejects/schedules meeting → System updates slot count.*
- *Faculty cancels approval → System restores slot availability.*

4.8.3 Functional Requirements

- ***4.8-1:*** *Faculty shall be able to accept or reject team requests.*
- ***4.8-2:*** *Faculty shall be able to schedule meetings with student teams.*
- ***4.8-3:*** *Slot availability shall update automatically after each action.*

4.9 Notification and Email Communication

4.9.1 Description and Priority

Description:
This feature enables the system to notify users through in-app mobile notifications, with the same notification content also sent via email for important updates and actions.

Priority: *Moderate-Low*

4.9.2 Stimulus / Response Sequence

System triggers an event → System sends a mobile notification → System sends the same notification content to the user's registered email address.

4.9.3 Functional Requirements

- ***4.9-1:*** *The system shall send mobile notifications for important events and updates.*
- ***4.9-2:*** *The system shall send the same notification content via email to the registered email address.*
- ***4.9-3:*** *The system shall not require users to access external links for notification delivery.*

4.10 Coordinator Dashboard & Allocation Reports

4.10.1 Description and Priority

Description:
Provides coordinators with dashboards to monitor allocation progress, unallocated students, and faculty slot utilization.

Priority: *Medium (Monitoring and decision support)*

4.10.2 Stimulus/Response Sequences

- *Coordinator opens dashboard → System displays real-time statistics.*
- *Allocation progresses → System updates report dynamically.*

4.10.3 Functional Requirements

- ***4.10-1:*** *The system shall display non allocated students.*
- ***4.10-2:*** *The system shall display faculty slot utilization.*
- ***4.10-3:*** *The system shall generate allocation reports.*

4.11 Administration & Record Management

4.11.1 Description and Priority

Description:
Supports administrative control for maintaining records and handling exceptional situations after deadlines.

Priority: *Medium (Administrative support)*

4.11.2 Stimulus/Response Sequences

- *Admin adds/removes users → System updates records.*
- *Admin changes coordinator role → System updates permissions.*

4.11.3 Functional Requirements

- ***4.11-1:*** *Admin shall be able to add or remove students and faculty.*
- ***4.11-2:*** *Admin shall be able to change faculty coordinator roles.*
- ***4.11-3:*** *The system shall maintain historical allocation records.*

5.Other Nonfunctional Requirements

5.1 Performance Requirements

- *The application should load the* ***dashboard/home screen*** *within* ***5 seconds*** *under a stable 4G or Wi-Fi connection.*
- *Project listings, faculty profiles, and slot availability details should be displayed within* ***5 seconds*** *after a user request.*
- *The system should support* ***simultaneous access by up to 1000 users*** *without significant performance degradation.*
- *Notifications such as* ***project request updates, approvals, rejections, and alerts*** *should be delivered within* ***4-5 seconds*** *of the triggering event.*
- *Critical operations such as* ***team formation, faculty request submission, approval actions, and slot updates*** *should be completed within* ***6–8 seconds****.*

Rationale

Slightly relaxed performance limits ensure stable operation under varying network conditions and peak academic usage periods, while still providing a responsive and reliable user experience for students, faculty, coordinators, and administrators.

5.2 Safety Requirements

- *The system shall prevent data loss or inconsistency during critical operations such as slot updates, faculty approvals, and team allocations.*
- *The application shall validate all inputs to prevent invalid team formation, exceeding slot limits, or rule violations defined by the Faculty Coordinator.*
- *The system shall ensure that slot availability updates occur atomically, so partial updates do not result in incorrect allocation states.*
- *In case of system failure, network interruption, or unexpected shutdown, the application shall ensure that previously saved data remains intact.*
- *The system shall provide confirmation prompts before performing irreversible actions such as request cancellation or record deletion.*
- *The application shall restrict access to critical operations (deadline overrides, coordinator role changes, record removal) to authorized users only.*
- *The system shall display clear error messages and guidance to prevent unsafe or unintended user actions.*

5.3 Security Requirements

- *The system shall require **secure user authentication** for all users (Students, Faculty, Faculty Coordinators, and Administrators) before granting access to any application features.*
- *The application shall implement **role-based access control (RBAC)** to ensure users can access only the data and functions permitted for their assigned role.*
- *All communication between the mobile application and backend services shall be secured using **HTTPS with SSL/TLS encryption** to protect data in transit.*
- *User credentials and authentication tokens shall be securely managed using **Firebase Authentication**, preventing unauthorized access.*
- *Sensitive user data such as **academic documents (marksheets, resumes)** shall be stored securely with controlled access and shall not be accessible to unauthorized users.*

- *The system shall maintain **audit logs** for important actions including slot updates, approvals, rejections, and role modifications.*
- *The application shall prevent unauthorized modification of **slot availability and allocation records**, ensuring data integrity at all times.*
- *Regular **automatic data backups** shall be performed to protect against data loss or system failure.*

5.4 Software Quality Attributes

- ***Availability:** The application shall maintain high availability (around **99.5% uptime**), excluding planned maintenance.*
- ***Reliability:** The system shall recover gracefully from crashes and preserve unsent actions.*
- ***Maintainability:** The system shall use a modular design to allow easy updates and bug fixes.*
- ***Usability:** The application shall be easy to use, enabling users to complete basic tasks with minimal effort.*
- ***Scalability:** The system shall support growth in users, institutions, and departments using a scalable cloud backend.*
- ***Portability:** The application shall run on Android devices and can be extended to other platforms with minimal changes.*
- ***Testability:** The system shall support automated testing to ensure correctness and stability.*

5.5 Business Rules

- *Only **verified users** can access the application based on assigned roles.*
- *Faculty members can supervise students **only within allocated slot limits**.*
- *Student teams can send requests **only within the defined deadline**.*
- *Slot availability shall update automatically after **approval or rejection**.*
- *Approved requests cannot be modified by students.*
- *Administrators can **add, remove, or modify users and roles** when required.*
- *All allocation activities shall be **logged for tracking and audit purposes**.*

6. Other Requirements

- ***Database Requirements:***
  The system shall use a centralized cloud database to store user profiles, project details, slot information, requests, and allocation records with proper backup and recovery mechanisms.

- ***Data Consistency Requirements:***
  Slot availability, approvals, and allocation records shall remain consistent across all users and institutions at all times.

- ***Internationalization:***
  The application shall be designed to support future localization (language and regional settings) if required.

- ***Legal and Compliance Requirements:***
  The system shall comply with institutional data protection policies and applicable data privacy regulations related to academic records.

- ***Reuse and Extensibility:***
  The system architecture shall support reuse of components and allow future extensions such as analytics, notifications, or additional roles without major redesign.

Appendix A: Glossary

- ***Admin (Administrator):***
  A user with the highest privileges responsible for managing users, roles, system configuration, and maintenance.

- ***Faculty Member:***
  A teacher who proposes project domains, manages student slots, and supervises allocated student teams.

- ***Faculty Coordinator:***
  A department-level authority responsible for defining project allocation rules, monitoring allocation progress, and resolving conflicts.

- ***Student (UG/PG):***
  Undergraduate or postgraduate student who forms a team, submits project requests, and participates in the project allocation process.

- ***Project Slot Capacity:***
  The maximum number of students or teams a faculty member can supervise during a semester.

- ***Project Allocation:***
  The process of assigning student teams to faculty-supervised projects based on slot availability and approvals.

- ***Team Leader:***
  A student designated to create and manage a team and submit requests on behalf of the team.

- ***Domain / Sub-Domain:***
  Academic areas of expertise selected by faculty members to categorize projects.

- ***FCM (Firebase Cloud Messaging):***
  A Firebase service used to send real-time mobile notifications to users.

- ***Flutter:***
  A cross-platform UI development framework used to build the mobile application.

- ***Firebase:***
  A backend platform used for authentication, database management, cloud storage, and notifications.

- ***SRS (Software Requirements Specification):***
  A document that formally defines the functional and non-functional requirements of the system.

- ***REST API:***
  Representational State Transfer Application Programming Interface used for client–server communication.

Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

Appendix C: To Be Determined List

- *Final list of **supported academic project domains and sub-domains** for each department.*

- *Maximum **student team size** policies across different institutions.*

- *Exact **allocation deadlines and semester timelines** for each institution.*
- *Rules for **project duration extensions** or early project termination.*
- *Institutional approval process for **exception handling and overrides**.*
- *Format and retention period for **audit logs and historical records**.*
- *Formation of Guides and Co-Guides from different departments for inter-department UG/PG Project*
- *Future integration requirements with **institutional ERP or academic systems** (if applicable).*